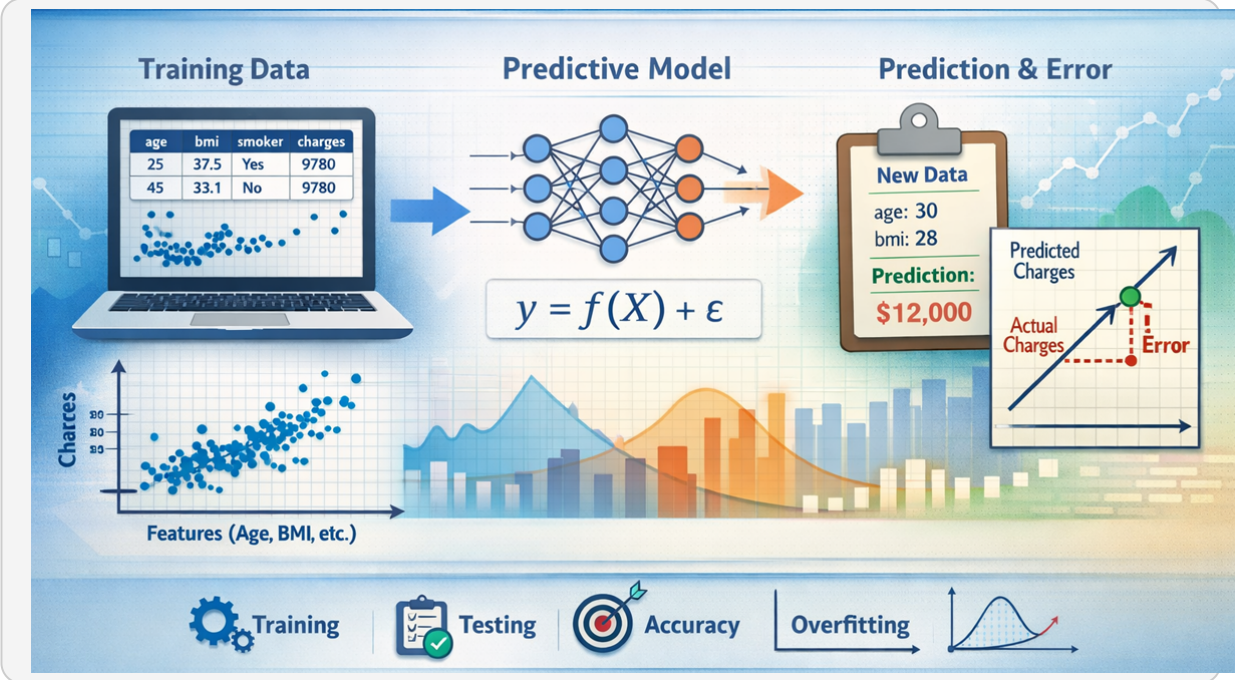
Chapter 11: MLR for Predictive Inference

<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to explain how the goals and evaluation criteria for predictive modeling differ from those of causal/explanatory modeling
<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to create train/test splits to measure out-of-sample generalization and prevent overfitting
<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to build reproducible preprocessing pipelines using scikit-learn that learn parameters from training data only
<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to fit linear regression models using scikit-learn and evaluate predictive performance using MAE and RMSE on held-out test data
<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to assess data readiness for prediction by checking missingness, identifying feature types, and preventing target leakage

11.1Introduction



In the previous two chapters, you learned how to build multiple linear regression (MLR) models and how to evaluate whether those models satisfy

the assumptions required for reliable *causal or explanatory interpretation*. In that setting, the primary goal was to understand *why* an outcome changes and how individual features contribute to that change, holding other factors constant.

In this chapter, we shift to a different—but equally important—goal: *prediction*. When prediction is the objective, the central question is no longer whether each coefficient can be interpreted causally. Instead, the focus is on how accurately a model can generate outcomes for new, unseen data.

Although both causal and predictive modeling often use the same mathematical tool—multiple linear regression—the *modeling mindset* changes substantially. Decisions about data cleaning, feature inclusion, assumption checking, and model evaluation are guided by predictive performance rather than statistical inference.

This chapter revisits linear regression from a predictive perspective. You will learn which assumptions matter less when prediction is the goal, which issues still threaten model reliability, and how concepts such as overfitting, generalization, and out-of-sample error shape modeling decisions.

We will also introduce a new modeling workflow centered on *scikit-learn*, the most widely used Python library for predictive machine learning. Unlike *statsmodels*, which emphasizes coefficient estimates and hypothesis tests, scikit-learn is designed around training models, generating predictions, and evaluating performance on held-out data.

Throughout the chapter, we will use the same *insurance* dataset you encountered in earlier chapters. This continuity allows you to directly compare causal and predictive modeling approaches using identical data, making the tradeoffs between interpretation and accuracy more concrete.

By the end of this chapter, you will be able to build a regression model optimized for predictive accuracy, evaluate its performance using appropriate metrics, and explain how and why predictive modeling decisions differ from those made in causal regression analysis.

11.2Causal to Prediction

In the previous two chapters, you learned how multiple linear regression can be used to *explain* relationships and support causal reasoning. In predictive modeling, the objective is different. The primary goal is not to understand why an outcome occurs, but to accurately estimate what the outcome will be for new, unseen observations.

When prediction is the goal, a model is evaluated by how well it *generalizes* beyond the data used to train it. A predictive model is considered successful if it produces low error on future data, even if its internal mechanics are difficult to interpret.

This shift in objective fundamentally changes the questions we ask during modeling:

- **Causal modeling asks:** Is this coefficient statistically significant? Can this effect be interpreted while holding other variables constant?
- **Predictive modeling asks:** Does this feature improve out-of-sample accuracy? Does removing it reduce prediction error?

Because of this difference, some concepts that were central in earlier chapters play a smaller role here. Statistical significance, p-values, and perfectly satisfied assumptions are no longer the primary criteria for success. What matters most is whether the model performs well on data it has never seen before.

This does not mean that predictive modeling ignores rigor or discipline. Instead, rigor is enforced through different mechanisms: careful data preparation, protection against information leakage, separation of training and testing data, and evaluation using appropriate error-based metrics.

In predictive regression, features are not included because they have a meaningful causal interpretation, but because they help the model make better predictions. A feature may be highly predictive even if its coefficient is unstable, correlated with other features, or difficult to explain in isolation.

Model complexity is also judged differently. In causal analysis, unnecessary complexity can obscure interpretation and weaken inference. In predictive modeling, additional complexity is acceptable as long as it improves generalization and does not lead to overfitting.

Table 11.1
Causal vs Predictive Regression: Key Differences

Dimension	Causal (Explanatory) Regression	Predictive Regression
Primary goal	Explain relationships and estimate effects	Accurately predict unseen outcomes
Key success metric	Statistical significance, confidence intervals	Out-of-sample error (e.g., MAE, RMSE)
Role of assumptions	Critical for valid inference	Secondary to predictive performance
Feature inclusion	Driven by theory and interpretability	Driven by contribution to accuracy
Model complexity	Minimized to preserve interpretability	Accepted if it improves generalization

Throughout this chapter, you will see that multiple linear regression can be used effectively as a predictive tool, even when some classical assumptions

are imperfectly satisfied. The emphasis shifts from “Is this model theoretically ideal?” to “Does this model reliably predict outcomes we care about?”

We will continue using the insurance dataset to illustrate these ideas. Rather than focusing on coefficient interpretation, we will focus on building, evaluating, and refining a model that predicts medical charges as accurately as possible.

Before building predictive models, however, we must revisit how data preparation changes when prediction—not explanation—is the primary objective.

11.3Data Preparation

When prediction is the goal, data preparation is evaluated by a single criterion: does this step help the model generalize to new, unseen data? This is a meaningful shift from causal regression, where preparation steps are often motivated by interpretability, assumption validity, or defensible inference.

In predictive regression, preprocessing decisions must also be *repeatable*. Any transformation applied during model development must be applied in exactly the same way when the model is used to make future predictions. This requirement will later motivate the use of pipelines, but we begin here with clear, explicit preparation steps using pandas.

We will continue using the insurance dataset introduced earlier in the book. For consistency across sections, we load the dataset once and reuse it throughout this chapter.

```
      import pandas as pd

      df = pd.read_csv(&amp;quot;/content/drive/MyDrive/Colab
Notebooks/data/insurance.csv&amp;quot;)
      df.head()
```

Before making any modeling decisions, predictive workflows begin with basic structural checks: dataset size, column names, data types, and a clear separation between the label and candidate predictors.

```
      display(df.info())

      # Separate label and predictors conceptually (no splitting yet)
      y = df[&amp;quot;charges&amp;quot;]
      X = df.drop(columns=[&amp;quot;charges&amp;quot;])

      # Output:
      # &amp;lt;class 'pandas.core.frame.DataFrame'&amp;gt;
      # RangeIndex: 1338 entries, 0 to 1337
      # Data columns (total 7 columns):
      #  #   Column    Non-Null Count  Dtype
      # ---  ------    --------------  -----
      #  0   age       1338 non-null   int64
      #  1   sex       1338 non-null   object
      #  2   bmi       1338 non-null   float64
      #  3   children  1338 non-null   int64
      #  4   smoker    1338 non-null   object
      #  5   region    1338 non-null   object
      #  6   charges   1338 non-null   float64
      # dtypes: float64(2), int64(2), object(3)
      # memory usage: 73.3+ KB
      # None
```

Unlike causal modeling, predictive modeling does not require us to justify each predictor theoretically. At this stage, we keep all available features and allow performance metrics later in the chapter to guide feature removal decisions.

Checking for missing values

Even when documentation suggests a dataset is complete, you should verify missingness directly. In predictive modeling, this step is not only diagnostic but operational: the same checks must hold when new data arrives in the future.

```
missing_by_column = X.isna().sum().sort_values(ascending=False)
print(missing_by_column)

print(&amp;quot;Any missing values?&amp;quot;, X.isna().any().any())

# Output:
# age         0
# sex         0
# bmi         0
# children    0
# smoker      0
# region      0
# dtype: int64
# Any missing values? False
```

This dataset contains no missing values, so no imputation is required for the walkthrough model. However, this verification step is still important because predictive pipelines must be robust to future data that may not be as clean as historical data.

Scaling considerations for prediction

In causal regression, scaling is often used to improve numerical stability or to interpret standardized coefficients. In predictive regression, scaling serves a different purpose: it ensures that features are on comparable numeric ranges for algorithms that are sensitive to feature magnitude.

At this stage, we do not apply scaling yet. Instead, we explicitly identify which columns are numeric and which are categorical. This classification will later allow us to apply different preprocessing steps to different feature types in a structured and repeatable way.

```
      numeric_features = X.select_dtypes(include=[&amp;quot;int64&amp;quot;,
&amp;quot;float64&amp;quot;]).columns
      categorical_features = X.select_dtypes(include=[&amp;quot;object&amp;quot;]).columns

      print(&amp;quot;Numeric features:&amp;quot;, list(numeric_features))
      print(&amp;quot;Categorical features:&amp;quot;, list(categorical_features))

      # Output:
      # Numeric features: ['age', 'bmi', 'children']
      # Categorical features: ['sex', 'smoker', 'region']
```

Leakage awareness

Information leakage occurs when a model is trained using information that would not be available at prediction time. Leakage can dramatically inflate apparent performance during development while producing poor results in real-world use.

The most obvious form of leakage is including the outcome variable as a predictor. By explicitly defining *charges* as the label and removing it from the feature matrix at the start of the workflow, we create a simple but effective guardrail against this error.

More subtle leakage risks arise when preprocessing decisions are informed by the full dataset rather than by training data alone. We will address those risks directly in the next section, where we introduce train/test splits and explain how they protect against overfitting.

Summary and Overview

At this point, we do not want to go too far into implementation details. Fully production-ready predictive pipelines rely on faster, more structured data representations than Pandas DataFrames, and we have not introduced those

tools yet. Instead, the goal here is to establish a *conceptual checklist*: the core data preparation considerations that must be addressed when building a predictive regression model, regardless of the specific tools used to implement them.

Table 11.2
Data preparation checklist for predictive regression

Preparation step	Key question	Predictive emphasis
Label definition	What am I trying to predict?	Explicitly separate the outcome from all predictors to prevent leakage.
Feature inclusion	Does this variable help prediction?	Keep features unless they hurt generalization; interpretability is secondary.
Missing values	Are values missing now or likely later?	Plan for consistent handling on future data, even if current data is clean.
Feature types	Which features are numeric vs categorical?	Classify early to support consistent preprocessing later.
Scaling	Will feature magnitudes affect learning?	Scaling is optional now, but must be consistent if used.
Leakage prevention	Am I using information unavailable at prediction time?	Guard against label leakage and “peeking” at future data.

11.4Regression Assumptions

The table below summarizes each of the regression assumptions and diagnostic tests including how they are relevant to predictive regression modeling.

Table 11.3

Regression assumptions in predictive modeling

Assumption	Why it mattered for causal inference	Why it matters for prediction	Typical predictive response
Linearity	Ensures coefficients represent valid average marginal effects and hypothesis tests are meaningful.	Signals missing structure that can cause systematic prediction error.	Add polynomial features, splines, or interaction terms.
Independence (no autocorrelation)	Required for unbiased standard errors and valid hypothesis tests.	Prevents overly optimistic estimates of generalization performance.	Use time-aware splitting, lag features, or specialized time-series models.
Homoscedasticity	Ensures standard errors and confidence intervals are correct.	Identifies regions of unreliable predictions and unstable error behavior.	Transform the label (e.g., log), apply weighted models, or segment data.
Normality of residuals	Supports valid t-tests and confidence intervals.	Mostly irrelevant for prediction accuracy.	Usually ignored unless extreme skew harms optimization.
No multicollinearity	Required for stable, interpretable coefficients and hypothesis testing.	Only problematic if it harms numerical stability or generalization.	Often keep correlated features; remove only if performance degrades.

Assumption	Why it mattered for causal inference	Why it matters for prediction	Typical predictive response
No outliers / influential points	Prevents distortion of coefficient estimates and inference.	Outliers may be informative or harmful depending on deployment context.	Evaluate impact on validation error before removing.

In earlier chapters, regression assumptions were introduced as requirements for valid inference and defensible causal interpretation. When prediction is the goal, those same assumptions do not disappear—but their purpose and priority change.

Rather than asking whether assumptions are satisfied to justify coefficient-level conclusions, predictive modeling asks a different question: do transformations inspired by these assumptions improve out-of-sample accuracy and generalization?

As a result, assumptions in predictive regression function less like strict rules and more like *diagnostic signals* that suggest potential feature engineering opportunities.

What matters less for prediction

Several assumptions that were critical for causal inference play a reduced role in predictive modeling. For example, residual normality is not required for accurate predictions, and multicollinearity is not inherently problematic as long as correlated features collectively improve predictive performance.

Similarly, unstable or difficult-to-interpret coefficients are acceptable in predictive regression. Features are retained or removed based on their

contribution to generalization error, not on whether their individual effects can be cleanly isolated.

What still matters for prediction

Other assumptions remain important because they directly affect model performance. Linearity of the relationship between features and the label matters insofar as violations indicate that the model is systematically missing structure that could be captured through transformation.

Heteroscedasticity also remains relevant, not because it biases coefficients, but because it signals uneven error variance that can reduce predictive reliability for certain regions of the feature space.

Autocorrelation continues to matter in predictive contexts involving time or sequence, because dependence between observations can cause models to overestimate their true generalization ability.

What changes meaning in predictive regression

In predictive modeling, assumptions are best understood as guides for feature engineering rather than criteria for model validity. Each assumption suggests specific transformations that may improve prediction, even if they complicate interpretation.

For example, nonlinearity motivates polynomial features and interaction terms, heteroscedasticity motivates variance-stabilizing transformations, and skewed distributions motivate logarithmic or other monotonic transforms.

Importantly, these transformations are not justified because they “fix” assumptions, but because they can reduce systematic error on unseen data.

Common predictive transformations motivated by diagnostics

Based on the diagnostics explored in the prior chapter, predictive regression commonly considers the following transformations: nonlinear feature expansions (such as squared terms), interaction effects between features, transformations of the label to stabilize variance, and retention of correlated features when they jointly improve prediction.

In the insurance dataset, this includes transformations such as squared age or BMI terms, interactions involving smoking status, and logarithmic transformations of medical charges to reduce skew and error heterogeneity.

We intentionally do not implement these transformations yet. In predictive modeling, feature engineering must be learned from training data and applied consistently to new data, which requires pipeline-based workflows that we introduce later in this chapter. The next sections focus on generalization, evaluation, and tooling before we build the full predictive model end to end.

11.5Train/Test Splits

In predictive modeling, the central question is not “How well does my model fit the data I already have?” but “How well will my model perform on new data I have never seen before?” This ability to perform well on unseen data is called *generalization*.

A model that fits the training data extremely well but performs poorly on new data is said to be *overfitting*. Overfitting occurs when the model learns noise, quirks, or accidental patterns in the training data rather than the true underlying signal.

To measure generalization directly, we must evaluate the model on data that was not used during training. This is accomplished by splitting the dataset into two parts:

- *Training set*: used to fit the model and learn preprocessing steps.
- *Test set*: held aside and used only for final evaluation.

In causal regression, analysts often fit models using the full dataset because the goal is coefficient estimation and inference. In predictive regression, however, fitting on all available data would prevent us from measuring whether the model truly generalizes.

From this point forward, all predictive models in this chapter will be trained using only the training set and evaluated using only the test set.

Creating a train/test split

We begin by loading the insurance dataset and separating the label (*charges*) from the predictor features. Then we create a train/test split using an 80/20 partition.

```
import pandas as pd
from sklearn.model_selection import train_test_split

# Separate label and predictors
y = df[&amp;quot;charges&amp;quot;]
X = df.drop(columns=[&amp;quot;charges&amp;quot;])

# Train/test split (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(
  X, y, test_size=0.2, random_state=42
)

print(&amp;quot;Training set size:&amp;quot;, X_train.shape)
print(&amp;quot;Test set size:&amp;quot;, X_test.shape)
```

The *random_state* parameter ensures that the split is reproducible, which is important for teaching, debugging, and fair model comparison.

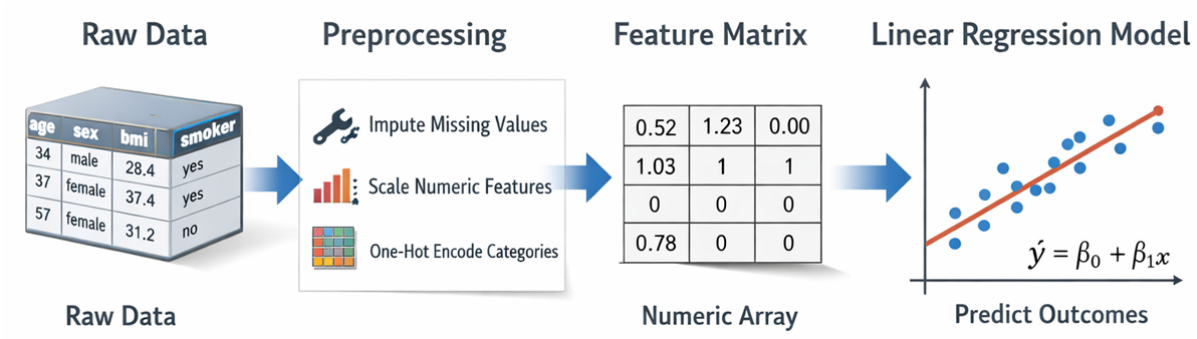
At this stage, no preprocessing has been performed. The data is still in raw Pandas DataFrame form. This is intentional: preprocessing must be learned from the training data only and applied consistently to the test data, which we will implement using pipelines in a later section.

These four objects will be reused throughout the remainder of the chapter:

- *X_train*: features used to train the model.
- *y_train*: labels used to train the model.
- *X_test*: features reserved for evaluation.
- *y_test*: true labels for evaluation.

In the next section, we will define how predictive performance is measured and why error-based metrics are more informative than traditional statistical measures such as p-values or R^2 when the goal is generalization.

11.6Pipelines in sklearn



Now that we have separated training and test data, the next step is to define how raw input features will be transformed into a numeric format suitable for

modeling. In predictive regression, these transformations must be applied *consistently* to training data, test data, and all future data seen in production.

In earlier chapters, we used Pandas functions such as *get_dummies()* and manual scaling to prepare data. While this approach works for causal analysis and small experiments, it does not scale well to real predictive systems because it is slow, error-prone, and difficult to reproduce exactly at inference time.

The sklearn library provides specialized objects for building fast, reliable, and reusable preprocessing workflows. These objects operate on NumPy arrays internally, which makes them much faster than DataFrame-based pipelines and suitable for large-scale machine learning systems.

Table 11.4
Key sklearn preprocessing objects used in predictive regression

Object	Purpose	Comparison to earlier chapters
train_test_split	Separates data into training and testing subsets to measure generalization and prevent overfitting.	Not required for causal modeling; essential for predictive modeling.
ColumnTransformer	Applies different preprocessing steps to different feature groups (numeric vs categorical).	Previously handled manually using Pandas column selection.
OneHotEncoder	Converts categorical variables into numeric indicator columns.	Replaces *pd.get_dummies()*; faster and safer for new/unseen categories.

Object	Purpose	Comparison to earlier chapters
StandardScaler	Standardizes numeric features to zero mean and unit variance.	Previously done manually or using ad-hoc transformations.
SimpleImputer	Fills missing values using learned statistics from training data.	Previously handled using DataFrame operations.
Pipeline	Chains preprocessing and modeling steps into a single reusable object.	New capability introduced for predictive workflows.

A major advantage of this approach is that preprocessing is *learned from training data only* and then reused automatically for the test set and future observations, preventing subtle forms of information leakage.

We will now construct a preprocessing pipeline for the insurance dataset using the training and test sets created in the previous section.

Defining numeric and categorical feature groups

We begin by identifying which columns are numeric and which are categorical in the training data. This ensures that transformations are learned only from the training distribution.

```
      # Identify feature types using training data only
      num_cols = X_train.select_dtypes(include=[&amp;quot;int64&amp;quot;,
&amp;quot;float64&amp;quot;]).columns
      cat_cols = X_train.select_dtypes(include=[&amp;quot;object&amp;quot;]).columns

      num_cols, cat_cols

      # Output:
      # (Index(['age', 'bmi', 'children'], dtype='object'),
      # Index(['sex', 'smoker', 'region'], dtype='object'))
```

Building the preprocessing components

Next, we define separate preprocessing steps for numeric and categorical features. Numeric features will be standardized, and categorical features will be one-hot encoded. Missing-value imputers are included for completeness, even though this dataset contains no missing values.

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer

numeric_preprocess = Pipeline(steps=[
  (&amp;quot;imputer&amp;quot;, SimpleImputer(strategy=&amp;quot;median&amp;quot;)),
  (&amp;quot;scaler&amp;quot;, StandardScaler())
])

categorical_preprocess = Pipeline(steps=[
  (&amp;quot;imputer&amp;quot;, SimpleImputer(strategy=&amp;quot;most_frequent&amp;quot;)),
  (&amp;quot;onehot&amp;quot;, OneHotEncoder(handle_unknown=&amp;quot;ignore&amp;quot;))
])
```

Combining transformations with ColumnTransformer

The ColumnTransformer applies the appropriate preprocessing pipeline to each group of features and concatenates the results into a single numeric matrix.

```
preprocessor = ColumnTransformer(
  transformers=[
    (&amp;quot;num&amp;quot;, numeric_preprocess, num_cols),
    (&amp;quot;cat&amp;quot;, categorical_preprocess, cat_cols)
  ]
)
```

Fitting preprocessing on training data only

We now fit the preprocessing pipeline using only the training data and apply it to both training and test sets.

```
X_train_ready = preprocessor.fit_transform(X_train)
X_test_ready = preprocessor.transform(X_test)

X_train_ready.shape, X_test_ready.shape

# Output:
# ((1070, 11), (268, 11))
```

Notice that the result is a NumPy matrix rather than a Pandas DataFrame. This representation is faster and is the standard input format expected by sklearn models.

At this point, our data is fully numeric, consistently scaled, safely encoded, and ready for model training.

In the next section, we will use these transformed features to train a predictive linear regression model using sklearn's modeling API.

11.7MLR in sklearn

In earlier chapters, you used Statsmodels to fit regression models designed for explanation and statistical inference. For predictive modeling, we shift to *scikit-learn (sklearn)*, a library optimized for performance, automation, and large-scale model deployment.

Sklearn treats regression as one component in a larger system: data preprocessing, feature transformation, model fitting, and prediction are designed to work together as a single pipeline.

This design reflects a fundamental difference in priorities: Statsmodels emphasizes coefficient interpretation and statistical testing, while sklearn emphasizes prediction accuracy, generalization, and repeatable workflows.

Key conceptual difference: Statsmodels fits models to DataFrames for human inspection; sklearn fits models to numeric matrices optimized for computation.

Table 11.5
Statsmodels vs scikit-learn for regression

Aspect	Statsmodels (causal)	scikit-learn (predictive)
Primary goal	Inference and explanation	Prediction and generalization
Input format	Pandas DataFrame	NumPy matrix
Preprocessing	Manual	Integrated via pipelines
Categorical encoding	get_dummies()	OneHotEncoder
Train/test workflow	Optional	Central
Statistical tests	Built-in	Not provided

Sklearn's linear regression model implements ordinary least squares just like Statsmodels, but it exposes only what is needed for prediction: coefficients, intercept, and prediction functions.

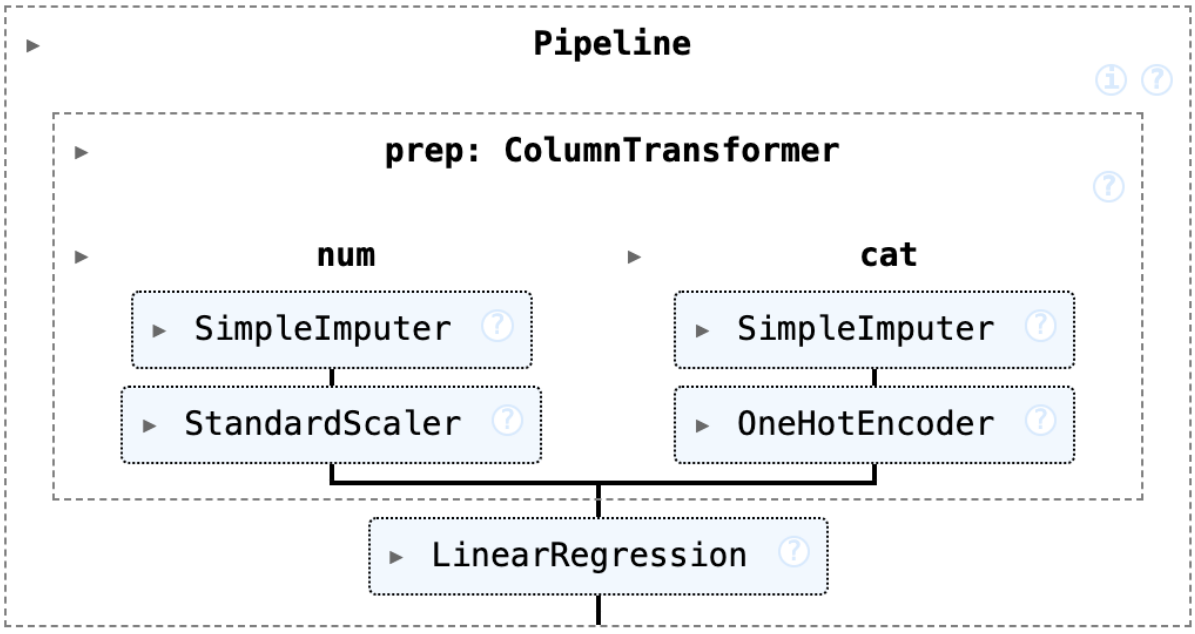
In predictive modeling, the regression model is rarely used alone. Instead, it is embedded inside a pipeline so that raw input data can be transformed and predicted in one consistent operation.

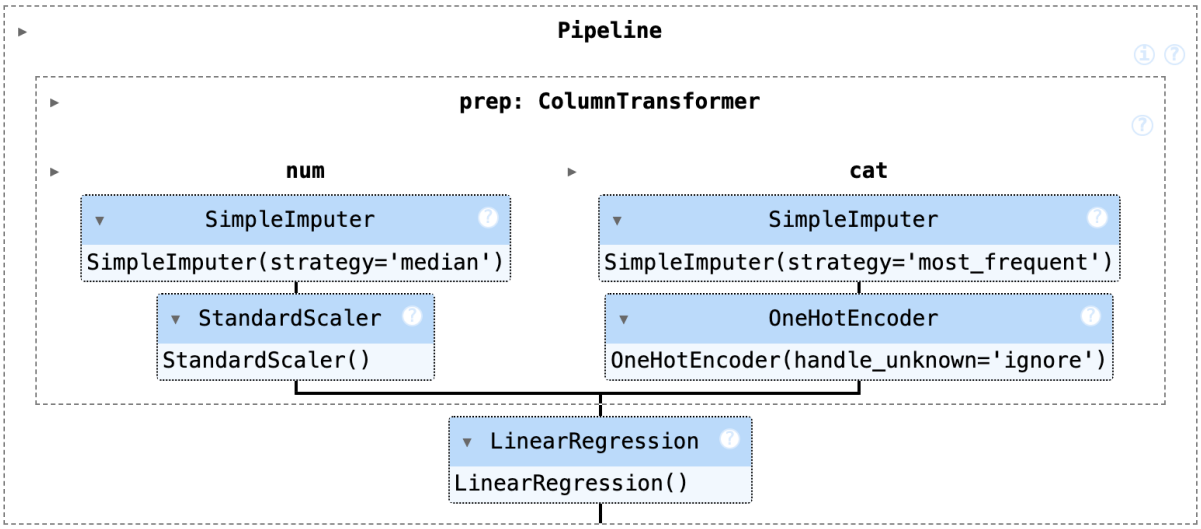
Adding a regression model to the preprocessing pipeline

We now extend the preprocessing pipeline created in the previous section by attaching a linear regression model as the final step.

This creates a complete predictive system that accepts raw insurance records and outputs predicted medical charges.

```
from sklearn.linear_model import LinearRegression

# Extend the existing preprocessing pipeline with a regression model
predictive_model = Pipeline(steps=[
  (&amp;quot;prep&amp;quot;, preprocessor),
  (&amp;quot;lr&amp;quot;, LinearRegression())
])

# Fit only on training data
predictive_model.fit(X_train, y_train)
```





At this point, the pipeline has learned two things from the training data: how to transform raw features into numeric form, and how to combine those features to predict charges.

No transformation parameters or regression coefficients were learned from the test data, preserving the integrity of future evaluation.

Once trained, the pipeline behaves like a single model object that can be used to generate predictions on any new dataset with the same structure.

```
      # Generate predictions for the test set
      y_pred = predictive_model.predict(X_test)

      # Preview first 5 predictions
      y_pred[:5]

      # Output:
      # array([ 8969.55027444, 7068.74744287, 36858.41091155, 9454.67850053,
26973.17345656])
```

The output is a NumPy array of predicted charges, one for each row in the test set (only the first five rows are displayed above because of the index [:5]).

In the next section, we will evaluate how accurate these predictions are using appropriate performance metrics for regression.

This completes the construction phase of our predictive regression pipeline: raw data → preprocessing → numeric matrix → trained model → predictions.

11.8Performance Metrics

In causal regression, model fit or quality is often discussed using statistical significance, confidence intervals, and R^2. In predictive modeling, these quantities are secondary. What matters most is how large the prediction errors are on new data.

For this reason, predictive regression models are evaluated using *error-based metrics* that directly measure how far predictions are from true outcomes.

Mean Absolute Error (MAE)

The *Mean Absolute Error (MAE)* is the average absolute difference between predicted values and actual values.

MAE is easy to interpret because it is expressed in the same units as the label. In this dataset, MAE is measured in dollars of medical charges.

Root Mean Squared Error (RMSE)

The *Root Mean Squared Error (RMSE)* squares errors before averaging and then takes the square root. This penalizes large mistakes more heavily than MAE.

RMSE is especially useful when large prediction errors are particularly costly or risky.

Choosing between MAE and RMSE

Although MAE and RMSE both measure prediction error in the same units as the label, they emphasize different types of mistakes.

MAE treats all errors equally and answers the question: “How wrong are we on average?” RMSE penalizes large errors more heavily and answers the question: “How severe are our worst mistakes?”

This difference matters in practice because different business problems care about different kinds of errors.

Table 11.6
When to emphasize MAE vs RMSE in practice

Business situation	Preferred metric	Reason
Customer billing estimates	MAE	Focus is on typical accuracy for most customers; extreme cases should not dominate evaluation.
Medical cost prediction (insurance)	Both	MAE reflects typical error; RMSE highlights risk from rare but extremely expensive patients.
Financial risk management	RMSE	Large prediction mistakes are disproportionately costly and must be strongly penalized.
Demand forecasting for inventory	RMSE	Large underestimates can cause stockouts and major revenue loss.

Business situation	Preferred metric	Reason
House price prediction for listings	MAE	Typical pricing accuracy matters more than rare extreme mispricing.
Energy load forecasting	RMSE	Large errors can cause grid instability or costly emergency generation.
Marketing response modeling	MAE	Individual errors are low-risk and should not be dominated by a few unusual customers.

In short, use MAE when typical accuracy is the goal and robustness to outliers is important. Use RMSE when large errors are especially dangerous or costly and should strongly influence model selection.

The limited role of R^2 in prediction

R^2 measures the proportion of variance explained in the dataset being evaluated. While useful for understanding model fit, it does not directly describe how large prediction errors are.

Two models can have similar R^2 values but very different MAE or RMSE values. For this reason, R^2 plays a supporting role in predictive modeling, while MAE and RMSE are primary.

A baseline for comparison

Before evaluating our trained model, we establish a simple baseline: always predicting the mean of the training labels. This represents a model that completely ignores all features and assumes every observation is “average.”

This baseline is not arbitrary. When prediction error is measured using squared error (and therefore RMSE), the mean is mathematically the optimal constant prediction—it minimizes expected squared error among all possible single-value predictions.

As a result, the mean predictor represents the best performance achievable without using any input features at all.

If a trained model cannot outperform this baseline on the test set, then the features and modeling process have added no predictive value, and the model has little practical usefulness.

Computing baseline performance

```
import numpy as np
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Baseline predictor: mean of training labels
baseline_value = y_train.mean()
y_pred_baseline = np.full_like(y_test, fill_value=baseline_value, dtype=float)

baseline_mae = mean_absolute_error(y_test, y_pred_baseline)
baseline_rmse = np.sqrt(mean_squared_error(y_test, y_pred_baseline))
baseline_r2 = r2_score(y_test, y_pred_baseline)

print(f&amp;quot;Baseline MAE:  {baseline_mae:,.2f}&amp;quot;)
print(f&amp;quot;Baseline RMSE: {baseline_rmse:,.2f}&amp;quot;)
print(f&amp;quot;Baseline R²:   {baseline_r2:.4f}&amp;quot;)

# Output:
# Baseline MAE:  9,593.34
# Baseline RMSE: 12,465.61
# Baseline R²:   -0.0009
```

Evaluating our predictive regression model

We now evaluate the predictions generated by the pipeline trained in the previous section.

```
model_mae = mean_absolute_error(y_test, y_pred)
model_rmse = mean_squared_error(y_test, y_pred, squared=False)
model_r2 = r2_score(y_test, y_pred)

print(f&amp;quot;Model MAE:  {model_mae:,.2f}&amp;quot;)
print(f&amp;quot;Model RMSE: {model_rmse:,.2f}&amp;quot;)
print(f&amp;quot;Model R²:   {model_r2:.4f}&amp;quot;)

# Output:
# Model MAE:  4,181.19
# Model RMSE: 5,796.28
# Model R²:   0.7836
```

As you can see, MAE, RMSE, and R^2 each improved dramatically indicating this model was worth generating.

Detecting overfitting through train/test comparison

After computing performance metrics on both training and test sets, the next critical step is comparing these metrics to assess whether the model generalizes well or is overfitting. Overfitting occurs when a model performs much better on training data than on test data, indicating it has learned training-specific patterns that do not generalize to new observations.

While the academic literature emphasizes qualitative assessment of the gap between training and test performance, practitioners often use quantitative heuristics to flag potential overfitting. These heuristics provide concrete thresholds that can be implemented in automated workflows, though they should be interpreted with domain knowledge and context rather than as absolute rules.

Table 11.7
Common heuristics for detecting overfitting in predictive regression

Heuristic	Threshold	Interpretation	Source/Context

Heuristic	Threshold	Interpretation	Source/Context
Test RMSE vs Train RMSE	Test RMSE > 10% higher than Train RMSE	If test RMSE exceeds training RMSE by more than 10%, the model may be overfitting. This indicates the model is memorizing training patterns rather than learning generalizable relationships.	Common industry practice; widely used in machine learning workflows (Hastie, Tibshirani, & Friedman, 2009; James et al., 2021)
R^2 gap	Training R^2 - Test R^2 > 0.10-0.15	A gap of 0.10 or more between training and test R^2 suggests the model explains substantially more variance in training data than it can generalize to new data.	Practical guideline; context-dependent based on problem complexity and data size
RMSE ratio	Test RMSE / Train RMSE > 1.2	When test RMSE is 20% or more higher than training RMSE, overfitting is likely. This ratio is unitless and works across different scales.	Alternative formulation of the 10% rule; provides relative rather than absolute comparison
Visual divergence	Widening gap in learning curves	When plotting training and test error across model complexity (e.g., number of features, tree depth), a widening gap indicates overfitting. The optimal model complexity is typically where test error is minimized before the gap widens.	Standard approach in machine learning (Hastie, Tibshirani, & Friedman, 2009; Murphy, 2022)

Heuristic	Threshold	Interpretation	Source/Context
Cross-validation consistency	High variance across folds	If performance varies substantially across cross-validation folds, the model may be overfitting to specific data partitions rather than learning stable patterns.	Cross-validation best practices (Kohavi, 1995; Arlot & Celisse, 2010)

It is important to note that these thresholds are practical heuristics rather than strict statistical rules. The appropriate threshold may vary depending on the problem domain, data size, model complexity, and business context. For example, in high-stakes applications like medical diagnosis, even small gaps between train and test performance may warrant concern, while in exploratory research, larger gaps might be acceptable. The key is to use these heuristics as diagnostic signals that prompt further investigation rather than as definitive pass/fail criteria.

When overfitting is detected, common remedies include reducing model complexity (fewer features, simpler algorithms), increasing regularization, collecting more training data, or using ensemble methods that combine multiple simpler models. The next section demonstrates one such approach: greedy backward feature removal, which systematically reduces model complexity to improve generalization.

At this stage, our model is functional but not yet optimized. Many features may contribute little to predictive accuracy or even harm generalization.

In the next section, we will refine the model using systematic feature engineering and removal to further improve out-of-sample performance.

Bibliography

The following references provide foundational coverage of overfitting detection, model evaluation, and cross-validation methods in machine learning and statistical learning theory.

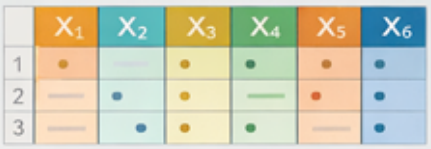
1. Arlot, S., & Celisse, A. (2010). A survey of cross-validation procedures for model selection. *Statistics Surveys*, 4, 40-79. https://doi.org/10.1214/09-SS054
2. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). Springer. https://doi.org/10.1007/978-0-387-84858-7
3. James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An Introduction to Statistical Learning: with Applications in R* (2nd ed.). Springer. https://doi.org/10.1007/978-1-0716-1418-1
4. Kohavi, R. (1995). A study of cross-validation and bootstrap for accuracy estimation and model selection. *Proceedings of the 14th International Joint Conference on Artificial Intelligence*, 2, 1137-1143. https://www.ijcai.org/Proceedings/95-2/Papers/016.pdf
5. Murphy, K. P. (2022). *Probabilistic Machine Learning: An Introduction*. MIT Press. https://probml.github.io/pml-book/book1.html

11.9Greedy Backward Feature Removal

Greedy Backward Feature Selection

Remove one feature at a time to see if the predictive model improves.
At each step, choose the single best feature to drop based on validation error.

Start with full predictor set



Initial model uses all columns

Which feature should we drop?

Test each removal on validation set

Greedy means we make the best choice we can right now, which might not be the best long-term combination.

Build & Validate Model

Which feature should we **drop**?

Measure error (ex: MAE, RMSE)

Test each removal on validation set

Test each removal on validation set

Test each removal on validation set

Repeat until stopping criteria met

- Prediction error (MAE, RMSE) has stopped improving
- You have reached a target number of features

Remove the feature that minimizes validation error

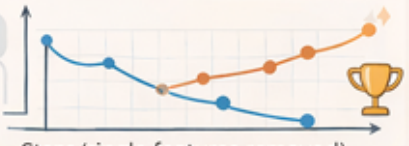
Initial model uses all columns

Dropped feature

- Prediction error (MAE, RMSE) has stopped improving
- You have reached a target number of features

Repeat until stopping criteria met

- Prediction error (MAE, RMSE) has stopped improving → You have reached a target number of features



Steps (single features removed)

In predictive regression, we choose features based on whether they improve prediction accuracy on new data. This section demonstrates a practical workflow called *greedy backward feature removal*, where we start with a full set of predictors and then remove one feature at a time while tracking how prediction error changes.

What “greedy” means

In optimization, a *greedy* method makes the best local choice available at each step, using the information it can measure *right now*. In our context, “best local choice” means: remove the single feature that produces the lowest validation error after removal (or, equivalently, produces the largest improvement in validation error) at that step.

Greedy methods are popular because they are straightforward and computationally manageable, but they are not guaranteed to find the absolute best subset of features. A greedy method does not test every possible combination of features; instead, it commits to one removal at a time, which can sometimes miss better combinations that only appear when removing multiple features together.

Why we use a validation set

To decide which feature to remove next, we need data that was not used to fit the model. That is the role of a *validation set*: a holdout sample used during model development to compare modeling choices, such as feature removal decisions.

A validation set is different from a test set. The *test set* is reserved for the final evaluation at the end of the workflow. If we repeatedly used the test set

to choose features, we would indirectly “train on the test set” by tailoring decisions to its outcomes, which inflates performance estimates and weakens the credibility of reported results.

In this section, we therefore split our original training data into two parts: a smaller *training subset* used to fit the model, and a *validation subset* used to evaluate feature removals. Only after we choose a stopping point will we evaluate the final model on the test set.

Feature engineering before feature selection

Before we begin removing features, we first expand the feature space using simple and interpretable *feature engineering*. This gives the model the opportunity to learn nonlinear patterns and subgroup-specific effects that a purely linear specification cannot capture.

Specifically, we create nonlinear transformations of *age* and *bmi* (such as squared and logarithmic terms) and interaction terms between smoking status and both the original and transformed variables. These engineered features act as additional candidate predictors that may or may not improve predictive accuracy.

In a predictive workflow, feature engineering and feature selection are tightly coupled: we generate potentially useful transformations first, then allow the greedy removal process to decide which of them are worth keeping based on validation-set error. Features that do not improve MAE or RMSE are treated as noise, even if they would be interesting to interpret in a causal analysis.

```
import numpy as np
import pandas as pd
```

```
      # ----------------------------------------------------------------------
      # Feature engineering: nonlinear terms + smoker interactions
      # ----------------------------------------------------------------------

      X_eng = X.copy()

      # 1) Create an explicit smoker indicator for interactions
      X_eng[&amp;quot;smoker_yes&amp;quot;] = (X_eng[&amp;quot;smoker&amp;quot;] ==
&amp;quot;yes&amp;quot;).astype(int)

      # 2) Nonlinear terms (examples used earlier in the book)
      X_eng[&amp;quot;age_sq&amp;quot;] = X_eng[&amp;quot;age&amp;quot;] ** 2

      # Use log(BMI). BMI is positive in this dataset; if you want to be extra safe:
      # X_eng[&amp;quot;bmi_ln&amp;quot;] = np.log(np.clip(X_eng[&amp;quot;bmi&amp;quot;], a_min=1e-6,
a_max=None))
      X_eng[&amp;quot;bmi_ln&amp;quot;] = np.log(X_eng[&amp;quot;bmi&amp;quot;])

      # 3) Interaction terms: smoker × age, smoker × bmi (and optionally with nonlinear
terms)
      X_eng[&amp;quot;age_x_smoker&amp;quot;] = X_eng[&amp;quot;age&amp;quot;] *
X_eng[&amp;quot;smoker_yes&amp;quot;]
      X_eng[&amp;quot;bmi_x_smoker&amp;quot;] = X_eng[&amp;quot;bmi&amp;quot;] *
X_eng[&amp;quot;smoker_yes&amp;quot;]

      # Interactions with nonlinear transforms (often useful)
      X_eng[&amp;quot;age_sq_x_smoker&amp;quot;] = X_eng[&amp;quot;age_sq&amp;quot;] *
X_eng[&amp;quot;smoker_yes&amp;quot;]
      X_eng[&amp;quot;bmi_ln_x_smoker&amp;quot;] = X_eng[&amp;quot;bmi_ln&amp;quot;] *
X_eng[&amp;quot;smoker_yes&amp;quot;]

      # Important: keep the original categorical columns too (sex, region, smoker)
      # because they may still carry predictive signal beyond interactions.
      X = X_eng
```

What we will build

- A loop that removes one feature at a time using a greedy rule based on validation-set error.
- A trace table that records which feature was removed at each step and how *MAE* and *RMSE* changed.
- A chart that plots MAE and RMSE as features are removed, with a label above each point showing the feature removed at that step.
- A stopping-criteria summary (evidence-based and domain-based) to guide where to stop removing features.

We will use the insurance dataset and the same preprocessing pipeline approach introduced earlier (scaling numeric features and one-hot encoding categorical features). The key difference is that we will now treat prediction error (MAE and RMSE) as the primary evidence for keeping or removing features.

Next, we will implement the greedy removal loop and generate the trace data that we will later visualize.

Greedy Backward Removal in Python

This code assumes you have already loaded the insurance dataset into a Pandas DataFrame named *df* and created *X* and *y* as shown earlier in the chapter (with *charges* as the label and all other columns as predictors).

It also assumes that the feature engineering step described above has already been applied, so that nonlinear and interaction features are included in *X* before feature selection begins.

We will use three datasets during development: (1) a training subset used to fit models, (2) a validation subset used to choose which feature to remove next, and (3) a test set reserved for later final evaluation. In this section, we create the validation split and generate the greedy-removal trace.

```
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error
```

```
      # Assumes these already exist from earlier code: df, y, X

      # 1) Hold out a final test set (not used for feature-removal decisions)
      X_train_full, X_test, y_train_full, y_test = train_test_split(
        X, y, test_size=0.20, random_state=42
      )

      # 2) Create a validation split from the training data (used for greedy decisions)
      X_train, X_val, y_train, y_val = train_test_split(
        X_train_full, y_train_full, test_size=0.25, random_state=42
      )
      # Note: 0.25 of the 0.80 training-full = 0.20 of the total dataset (roughly 60/20/20
split overall)

      # 3) Identify numeric vs categorical columns (insurance has both)
      num_cols = X_train.select_dtypes(include=[&quot;int64&quot;,
&quot;float64&quot;]).columns.tolist()
      cat_cols = X_train.select_dtypes(include=[&quot;object&quot;]).columns.tolist()

      # 4) Define preprocessing templates
      numeric_preprocess = Pipeline(steps=[
        (&quot;imputer&quot;, SimpleImputer(strategy=&quot;median&quot;)),
        (&quot;scaler&quot;, StandardScaler())
      ])

      categorical_preprocess = Pipeline(steps=[
        (&quot;imputer&quot;, SimpleImputer(strategy=&quot;most_frequent&quot;)),
        (&quot;onehot&quot;, OneHotEncoder(handle_unknown=&quot;ignore&quot;))
      ])

      # 5) Helper: build a full pipeline for a given subset of raw input columns
      def make_model(selected_features):
        selected_num = [c for c in selected_features if c in num_cols]
        selected_cat = [c for c in selected_features if c in cat_cols]

        preprocessor = ColumnTransformer(
          transformers=[
            (&quot;num&quot;, numeric_preprocess, selected_num),
            (&quot;cat&quot;, categorical_preprocess, selected_cat)
          ],
          remainder=&quot;drop&quot;
        )

        model = Pipeline(steps=[
          (&quot;prep&quot;, preprocessor),
          (&quot;lr&quot;, LinearRegression())
        ])

        return model

      # 6) Helper: fit on training subset and score on validation subset
      def fit_and_score(selected_features):
        m = make_model(selected_features)
        m.fit(X_train[selected_features], y_train)

        y_hat = m.predict(X_val[selected_features])
        mae = mean_absolute_error(y_val, y_hat)
        rmse = mean_squared_error(y_val, y_hat, squared=False)

        return mae, rmse

      # 7) Greedy backward removal trace
      all_features = X_train.columns.tolist()
```

```
      current_features = all_features.copy()

      trace_rows = []

      # Baseline (no removals yet)
      base_mae, base_rmse = fit_and_score(current_features)
      trace_rows.append({
        &quot;step&quot;: 0,
        &quot;removed_feature&quot;: &quot;(none)&quot;,
        &quot;n_features&quot;: len(current_features),
        &quot;val_mae&quot;: base_mae,
        &quot;val_rmse&quot;: base_rmse,
        &quot;remaining_features&quot;: current_features.copy()
      })

      for step in range(1, len(all_features)):
        best_candidate = None
        best_mae = None
        best_rmse = None

        for f in current_features:
          candidate_features = [c for c in current_features if c != f]
          mae, rmse = fit_and_score(candidate_features)

          if (best_rmse is None) or (rmse &lt; best_rmse) or (rmse == best_rmse and mae
&lt; best_mae):
            best_candidate = f
            best_mae = mae
            best_rmse = rmse

        current_features.remove(best_candidate)

        trace_rows.append({
          &quot;step&quot;: step,
          &quot;removed_feature&quot;: best_candidate,
          &quot;n_features&quot;: len(current_features),
          &quot;val_mae&quot;: best_mae,
          &quot;val_rmse&quot;: best_rmse,
          &quot;remaining_features&quot;: current_features.copy()
        })

      trace = pd.DataFrame(trace_rows)
      display(trace)
```

	step	removed_feature	n_features	val_mae	val_rmse	remaining_features
0	0	(none)	13	3173.648550	5604.757681	[age, sex, bmi, children, smoker, region, smok...
1	1	bmi_x_smoker	12	3151.218884	5589.645457	[age, sex, bmi, children, smoker, region, smok...
2	2	age_x_smoker	11	3150.961864	5588.050141	[age, sex, bmi, children, smoker, region, smok...
3	3	age	10	3156.677606	5580.853100	[sex, bmi, children, smoker, region, smoker_ye...
4	4	age_sq_x_smoker	9	3155.502133	5579.965205	[sex, bmi, children, smoker, region, smoker_ye...
5	5	smoker_yes	8	3155.502133	5579.965205	[sex, bmi, children, smoker, region, age_sq, b...
6	6	bmi	7	3154.573745	5580.593671	[sex, children, smoker, region, age_sq, bmi_ln...
7	7	bmi_ln	6	3159.451484	5579.906675	[sex, children, smoker, region, age_sq, bmi_ln...
8	8	sex	5	3166.589869	5586.129272	[children, smoker, region, age_sq, bmi_ln_x_sm...
9	9	region	4	3163.011018	5607.618157	[children, smoker, age_sq, bmi_ln_x_smoker]
10	10	children	3	3237.744866	5675.473234	[smoker, age_sq, bmi_ln_x_smoker]
11	11	smoker	2	4168.963199	6673.469826	[age_sq, bmi_ln_x_smoker]
12	12	age_sq	1	5864.691056	7820.519086	[bmi_ln_x_smoker]

The trace table records the validation-set error after each greedy removal step. Step 0 is the baseline (all features). Step 1 removes one feature and records the resulting MAE and RMSE. The process continues until only one feature remains.

Notice that this method is computationally expensive because it refits many models: at each step, it tries removing every remaining feature and chooses the best local option. Even so, this is still far more feasible than testing every possible subset of features, which grows exponentially as the number of features increases.

In the next section, we will visualize the trace by plotting MAE and RMSE across steps and labeling each point with the feature removed at that step. This makes it easier to choose a reasonable stopping point based on evidence (error changes) and domain knowledge (which features were removed).

Visualizing MAE and RMSE Across Removals

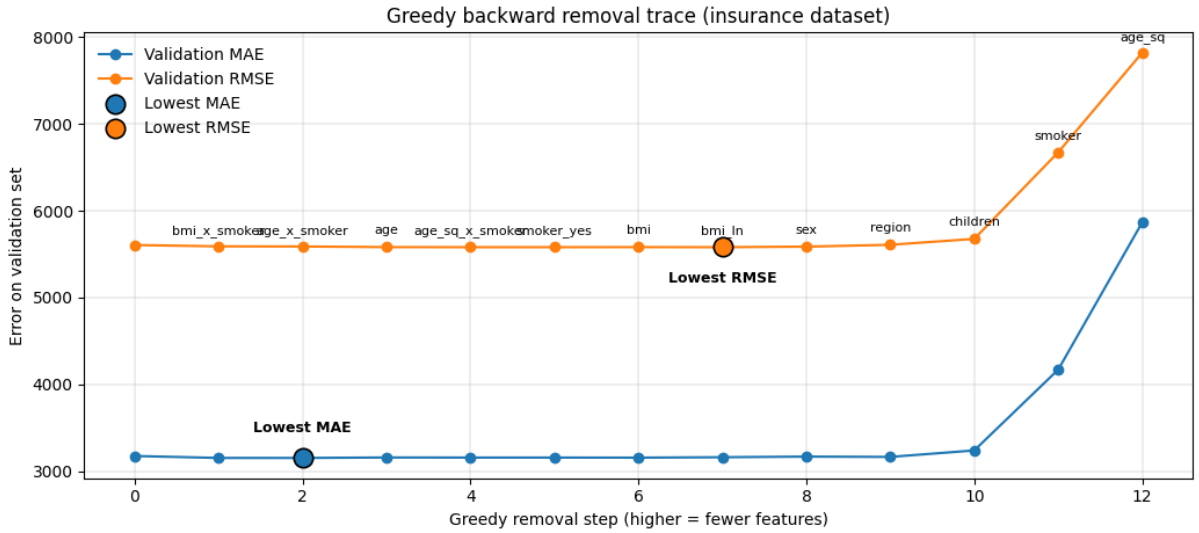
The trace table is useful, but the key idea is easier to see visually. In a predictive workflow, we often look for a point where removing additional features provides little benefit (or begins to harm performance). This is sometimes called an “elbow” in the error curve.

In the plot below, each point represents one step in the greedy backward removal process. The y-values show validation-set error (MAE and RMSE). The label above each point shows the feature that was removed to reach that step.

Why include feature names on the plot? Because stopping decisions are not only statistical; they are also practical. If the next feature to remove is something you believe is essential for real-world prediction (based on domain knowledge), you might stop earlier—even if the curve suggests small gains from removing it.

```
      import matplotlib.pyplot as plt

      # Assumes you already ran the prior chunk and have:
      # trace (DataFrame) with columns: step, removed_feature, n_features, val_mae,
val_rmse

      # X-axis is the step number (0 = full model, then one feature removed each step)
      x = trace[&quot;step&quot;].to_numpy()
      mae = trace[&quot;val_mae&quot;].to_numpy()
      rmse = trace[&quot;val_rmse&quot;].to_numpy()
      labels = trace[&quot;removed_feature&quot;].tolist()

      plt.figure(figsize=(10.5, 4.8))

      # Plot MAE and RMSE as separate lines
      plt.plot(x, mae, marker=&quot;o&quot;, linewidth=1.5, label=&quot;Validation
MAE&quot;)
      plt.plot(x, rmse, marker=&quot;o&quot;, linewidth=1.5, label=&quot;Validation
RMSE&quot;)

      plt.xlabel(&quot;Greedy removal step (higher = fewer features)&quot;)
      plt.ylabel(&quot;Error on validation set&quot;)
      plt.title(&quot;Greedy backward removal trace (insurance dataset)&quot;)
      plt.legend(frameon=False)

      # Add feature-name labels above each point (skip step 0 label since nothing was
removed)
```

```
for i in range(1, len(x)):
  # RMSE labels (place above RMSE point)
  plt.annotate(
    labels[i],
    (x[i], rmse[i]),
    textcoords=&amp;quot;offset points&amp;quot;,
    xytext=(0, 8),
    ha=&amp;quot;center&amp;quot;,
    fontsize=8
  )

# Light grid for readability
plt.grid(True, linewidth=0.3)
plt.tight_layout()
plt.show()
```



The highlighted points show the lowest validation MAE and lowest validation RMSE observed during the greedy removal process. In this run, MAE reaches its minimum early, while RMSE reaches its minimum several steps later after additional features have been removed.

This pattern is common in practice. MAE often improves quickly and then flattens, while RMSE may continue to improve slightly as the model becomes simpler and large errors are reduced. Because RMSE penalizes large mistakes more heavily, it often favors a more conservative model with fewer features.

If your application places high cost on large prediction errors (for example, forecasting medical expenses, insurance risk, or financial losses), you may prioritize the stopping point near the minimum RMSE. If interpretability and average accuracy are more important, you may prefer a stopping point near the minimum MAE.

In this example, both curves remain relatively flat for several steps before rising sharply once too many features have been removed. This flat region represents a practical “sweet spot” where the model is simpler but still highly accurate. A reasonable stopping point would lie somewhere in this region, before the sharp increase in error.

Rather than selecting the single absolute minimum mechanically, predictive modeling often uses this plot to guide judgment: choose the simplest model that achieves near-minimum error while avoiding the steep degradation that signals underfitting.

In the next section, we formalize several stopping criteria—both evidence-based (using changes in MAE and RMSE) and domain-based (using knowledge about which features are meaningful or stable)—and show how to produce a final selected feature set.

Choosing a Stopping Point and Finalizing the Feature Set

A greedy backward procedure always produces a complete removal path: a full model, then a model with one feature removed, then two removed, and so on until only one feature remains. The remaining question is: *where should you stop?*

In predictive modeling, stopping is not based on statistical significance. Instead, stopping is based on a tradeoff between (1) predictive accuracy on

new data and (2) simplicity and robustness. Removing features can sometimes reduce overfitting and improve generalization, but removing too many features eventually harms accuracy.

What is a validation set, and why did we include it?

A *validation set* is a subset of data that is *not used to train the model* but is used repeatedly to guide modeling decisions—such as selecting features. We used a validation set so that the greedy procedure could compare different feature sets using a consistent, held-out dataset.

This prevents a common mistake: selecting features using the same data that was used to fit the model. If you evaluate candidate feature sets on training data, you will often select a model that looks great in development but performs worse on truly new data. Using a validation set reduces this risk.

Later in the book, you will learn stronger methods such as cross-validation, which reduce sensitivity to a single validation split. For now, a train/validation/test structure is a practical and understandable foundation.

Stopping criteria

There is no universal “best” stopping rule. In practice, modelers use a combination of evidence-based criteria (MAE/RMSE changes) and judgment-based criteria (domain knowledge, operational constraints). The table below summarizes common approaches.

Table 11.8
Stopping criteria for greedy backward feature removal

Stopping criterion	How it works	When it is useful

Stopping criterion	How it works	When it is useful
Minimum validation MAE	Stop at the step with the lowest MAE on the validation set.	When average-size errors matter most and interpretability of the metric is important.
Minimum validation RMSE	Stop at the step with the lowest RMSE on the validation set.	When large errors are especially costly or risky and you want to penalize big misses.
Elbow / diminishing returns	Stop when MAE/RMSE improvements become small from one step to the next.	When you want a simpler model that is “almost as good” as the best-performing model.
Percent-change threshold	Stop when the improvement in MAE/RMSE is less than a chosen percentage (for example, < 1%) compared to the previous step.	When you want a reproducible rule that can be explained and applied consistently.
Keep N features	Stop when a specified number of features remain (for example, keep 8 predictors).	When deployment constraints require a simple model or limited data collection.
Domain knowledge override	Stop early if the next feature to remove is considered essential, stable, or required for business reasons.	When interpretability, policy constraints, or operational realities outweigh small error improvements.

Producing a final selection using the trace

Because we already computed the full greedy trace, selecting a final model is primarily a matter of choosing the step you want. In the code below, we show

two simple ways to choose a stopping point: (1) keep a fixed number of features, or (2) stop when improvement falls below a chosen threshold.

```
      import numpy as np

      # Assumes: trace (DataFrame) exists from the greedy procedure
      # Columns: step, removed_feature, n_features, val_mae, val_rmse, remaining_features

      # --- Option A: Keep a fixed number of features ---
      def choose_step_keep_n(trace_df, n_keep):
        # Find the row where n_features == n_keep (closest if not exact)
        idx = (trace_df[&quot;n_features&quot;] - n_keep).abs().idxmin()
        return int(trace_df.loc[idx, &quot;step&quot;])

      # --- Option B: Percent-change threshold on MAE (similar can be done for RMSE) ---
      def choose_step_by_threshold(trace_df, metric_col=&quot;val_mae&quot;,
min_improvement_pct=1.0):
        # Improvement is measured as percent decrease from previous step to current step
        vals = trace_df[metric_col].to_numpy()
        # Start at step 1 because step 0 has no previous comparison
        for i in range(1, len(vals)):
          prev = vals[i - 1]
          curr = vals[i]
          # Percent improvement (positive means error decreased)
          pct_improve = (prev - curr) / prev * 100.0
          if pct_improve &lt; min_improvement_pct:
            # Stop at the previous step (last meaningful improvement)
            return i - 1
        # If we never drop below threshold, stop at the best (lowest) metric
        return int(trace_df[metric_col].idxmin())

      # Choose a stop in either way (examples)
      stop_step_a = choose_step_keep_n(trace, n_keep=8)
      stop_step_b = choose_step_by_threshold(trace, metric_col=&quot;val_rmse&quot;,
min_improvement_pct=0.01)

      print(&quot;Stop step (keep N):&quot;, stop_step_a)
      print(&quot;Stop step (threshold):&quot;, stop_step_b)

      # Pull the selected features from the trace
      selected_features_a = trace.loc[trace[&quot;step&quot;] == stop_step_a,
&quot;remaining_features&quot;].iloc[0]
      selected_features_b = trace.loc[trace[&quot;step&quot;] == stop_step_b,
&quot;remaining_features&quot;].iloc[0]

      print(&quot;Selected features (keep N):&quot;, selected_features_a)
      print(&quot;Selected features (threshold):&quot;, selected_features_b)

      # Output:
      # Stop step (keep N): 5
      # Stop step (threshold): 4
      # Selected features (keep N): ['sex', 'bmi', 'children', 'smoker', 'region',
'age_sq', 'bmi_ln', 'bmi_ln_x_smoker']
      # Selected features (threshold): ['sex', 'bmi', 'children', 'smoker', 'region',
'smoker_yes', 'age_sq', 'bmi_ln', 'bmi_ln_x_smoker']
```

Fit the final model and store it as *final_model*

Up to this point, we have trained many temporary models to compare feature removals, but we have not saved a single “final” model object. In the code below, we (1) choose a stopping step, (2) extract the remaining features at that step, and (3) refit one pipeline on the combined training+validation data. We store that trained pipeline as *final_model*, which we will reuse in the next section to make out-of-sample predictions.

```
      # Assumes you already ran earlier code and have: trace, X_train_full, X_test,
y_train_full, y_test, and make_model()

      stop_step = choose_step_keep_n(trace, n_keep=8)
      selected_features = trace.loc[trace[&amp;quot;step&amp;quot;] == stop_step,
&amp;quot;remaining_features&amp;quot;].iloc[0]

      final_model = make_model(selected_features)
      final_model.fit(X_train_full[selected_features], y_train_full)

      y_pred = final_model.predict(X_test[selected_features])

      mae = mean_absolute_error(y_test, y_pred)
      rmse = np.sqrt(mean_squared_error(y_test, y_pred))

      print(&amp;quot;Stop step:&amp;quot;, stop_step)
      print(&amp;quot;Selected features:&amp;quot;, selected_features)
      print(&amp;quot;Test MAE:&amp;quot;, round(mae, 2))
      print(&amp;quot;Test RMSE:&amp;quot;, round(rmse, 2))
```

The key output from this cell is the trained pipeline stored in *final_model* and the list of *selected_features*. In the next section, we will create a one-row DataFrame with those same feature names and call *final_model.predict()* to generate a prediction for a brand-new case.

In this chapter, we used a validation set specifically to guide feature removal decisions. That means test-set evaluation should be saved for the end, after you commit to a final model.

The greedy backward approach used here is intentionally simple and transparent, making it ideal for learning the mechanics of predictive feature selection. However, it is not the most robust method available. In later chapters, you will improve this workflow using cross-validation and more reliable feature importance techniques such as *Permutation Feature Importance (PFI)*, which estimates a feature's contribution by measuring how much prediction error increases when that feature is randomly shuffled. These methods provide stronger evidence of true predictive value, especially in high-dimensional or highly correlated datasets. For now, the key takeaway is that predictive feature selection should be guided by *out-of-sample error behavior*, not by p-values, and that this chapter's approach should be viewed as a conceptual foundation rather than a final best practice.

11.10Making Predictions

After you train a predictive model, the next practical question is: how do you use it to make predictions for a new, unseen case? In scikit-learn, this is done with the *predict()* method.

The most important idea is that you should not manually redo your preprocessing steps. Instead, you know your model is a pipeline (preprocessing + regression), so you can pass raw inputs in the same format as your original *X* matrix and let the pipeline apply the exact same transformations automatically.

Why pipelines matter for prediction

When you trained your pipeline, it learned everything needed to transform raw inputs into the model's internal feature representation. That includes numeric imputation and scaling, categorical encoding, and any other

preprocessing steps you included. If you skip the pipeline and try to transform inputs manually, it is easy to apply slightly different rules and accidentally produce inconsistent predictions.

Prediction workflow

- Collect inputs for a single new case (for example, age, BMI, smoker status, etc.).
- Store those inputs in a one-row Pandas DataFrame with the same column names as the training *X*.
- Call *pipeline.predict(new_case_df)* to generate the prediction.

Example: interactive input and prediction (console)

The code below demonstrates a simple way to collect user input, build a one-row DataFrame, and generate a prediction. It assumes you already trained a pipeline named *final_model* (the same kind of pipeline used in this chapter), and that the pipeline expects the same raw columns that exist in *X*.

```
      import pandas as pd

      # Assumes you already trained this earlier in the section:
      # final_model = Pipeline(steps=[(&amp;quot;prep&amp;quot;, preprocessor), (&amp;quot;lr&amp;quot;,
LinearRegression())])
      # and that your training features were stored as:
      # feature_cols = X.columns.tolist()

      def get_float(prompt):
        return float(input(prompt))

      def get_int(prompt):
        return int(input(prompt))

      def get_str(prompt):
        return input(prompt).strip()

      # Example (insurance-style) inputs; adjust these prompts to match your dataset
columns
```

```
      age = get_int(&amp;quot;Enter age (e.g., 37): &amp;quot;)
      bmi = get_float(&amp;quot;Enter BMI (e.g., 28.5): &amp;quot;)
      children = get_int(&amp;quot;Enter number of children (e.g., 2): &amp;quot;)
      sex = get_str(&amp;quot;Enter sex (male/female): &amp;quot;)
      smoker = get_str(&amp;quot;Smoker? (yes/no): &amp;quot;)
      region = get_str(&amp;quot;Enter region (northeast/northwest/southeast/southwest):
&amp;quot;)

      new_case = pd.DataFrame([{
        &amp;quot;age&amp;quot;: age,
        &amp;quot;bmi&amp;quot;: bmi,
        &amp;quot;children&amp;quot;: children,
        &amp;quot;sex&amp;quot;: sex,
        &amp;quot;smoker&amp;quot;: smoker,
        &amp;quot;region&amp;quot;: region
      }])

      # If your X included engineered features, rebuild them here in the same way
      # (Only needed if engineered columns were explicitly added to X before training.)
      new_case[&amp;quot;smoker_yes&amp;quot;] = (new_case[&amp;quot;smoker&amp;quot;] ==
&amp;quot;yes&amp;quot;).astype(int)
      new_case[&amp;quot;age_sq&amp;quot;] = new_case[&amp;quot;age&amp;quot;] ** 2
      new_case[&amp;quot;bmi_ln&amp;quot;] = (new_case[&amp;quot;bmi&amp;quot;]).apply(lambda v:
__import__(&amp;quot;numpy&amp;quot;).log(v))
      new_case[&amp;quot;age_x_smoker&amp;quot;] = new_case[&amp;quot;age&amp;quot;] *
new_case[&amp;quot;smoker_yes&amp;quot;]
      new_case[&amp;quot;bmi_x_smoker&amp;quot;] = new_case[&amp;quot;bmi&amp;quot;] *
new_case[&amp;quot;smoker_yes&amp;quot;]
      new_case[&amp;quot;age_sq_x_smoker&amp;quot;] = new_case[&amp;quot;age_sq&amp;quot;] *
new_case[&amp;quot;smoker_yes&amp;quot;]
      new_case[&amp;quot;bmi_ln_x_smoker&amp;quot;] = new_case[&amp;quot;bmi_ln&amp;quot;] *
new_case[&amp;quot;smoker_yes&amp;quot;]

      pred = final_model.predict(new_case)[0]
      print(&amp;quot;Predicted value:&amp;quot;, round(pred, 2))
```

How this connects to deployment

This interactive example is not meant to be a final user interface. Its purpose is to show the mechanics: collect raw inputs, create a one-row DataFrame with matching column names, and pass it into *predict()*. Later, when you deploy models into apps or websites, the same logic still applies—your app will gather inputs through a form instead of *input()*, but the pipeline will still transform and predict in exactly the same way.

Practical tip: Save the list of training columns (for example, *feature_cols = X.columns.tolist()*) so you can quickly verify that your prediction DataFrame

has the same columns. A mismatch in column names is one of the most common causes of prediction errors during deployment.

11.11Case Studies

Try the practice problems below to see how well you understand the chapter content.

Case #1: Diamonds Dataset
This practice uses the **Diamonds** dataset that ships with the Seaborn Python package. Your goal is to build a **predictive** regression model that estimates diamond *price* as accurately as possible on new data. Unlike causal modeling, you will evaluate success using *MAE* and *RMSE* on holdout data and you will use a *greedy backward feature removal* workflow to reduce overfitting.

Dataset attribution: The Diamonds dataset is distributed with the Seaborn data repository and can be loaded with *seaborn.load_dataset("diamonds")*. If you want the underlying CSV source, Seaborn hosts it in its public GitHub repository under *seaborn-data*.

To load the dataset, use this code:

```
import pandas as pd
import seaborn as sns

df = sns.load_dataset(&amp;quot;diamonds&amp;quot;)
df.head()
```

Prediction goal: Predict *price* using a mix of numeric features (*carat*, *depth*, *table*, *x*, *y*, *z*) and categorical features (*cut*, *color*, *clarity*). Use a predictive workflow: train/validation/test splits, preprocessing in an sklearn pipeline, and error-based evaluation.

Tasks

- Inspect the dataset: rows/columns, data types, and summary statistics for *price*.
- Create *X* and *y* where *y = price* and *X* includes the predictors listed above. Do not use any columns that trivially reveal the label (none should in this dataset), and document your chosen feature set.
- Split your data into *train*, *validation*, and *test* sets (roughly 60/20/20 overall). The validation set will be used for greedy feature-removal decisions, and the test set must be held until the end.
- Build an sklearn preprocessing pipeline that (a) scales numeric predictors (StandardScaler) and (b) one-hot encodes categorical predictors (OneHotEncoder with *handle_unknown="ignore"*). Fit preprocessing only on training data via the pipeline to prevent leakage.
- Establish a baseline error on the validation set by predicting the *training mean* of *price*. Report baseline *MAE* and *RMSE*.
- Fit a baseline predictive linear regression model (with all features) using your pipeline. Report validation *MAE*, validation *RMSE*, and test-set metrics only at the end.
- **Feature engineering (before selection):** Create at least two interpretable nonlinear numeric features (for example, *carat_sq* and *log_carat*, or *x_sq* and *y_sq*). If you create a log term, ensure the input is positive (use clipping if needed). Add these engineered features to *X* before feature selection.
- Run *greedy backward feature removal* using the validation set: at each step, try removing each remaining feature one-at-a-time, refit the model, and choose the removal that yields the lowest validation *RMSE* (use MAE as a tie-breaker). Record a trace table with *step*, *removed_feature*, *n_features*, *val_mae*, and *val_rmse*.

- Create a line plot of validation *MAE* and *RMSE* across greedy steps. Add a label above each point showing which feature was removed at that step. Highlight the lowest MAE point and the lowest RMSE point to make them easy to identify.
- Choose a stopping point using one evidence-based rule (for example, keep N features, or stop when improvement falls below a percent threshold) and one domain/judgment-based reason (for example, "we stopped before removing *carat* because it is core to the pricing mechanism"). Document your final selected feature set.
- Refit the final model using the selected features (training + validation can be recombined after the decision). Evaluate once on the untouched *test* set and report test *MAE* and test *RMSE*.

Analytical questions (answers should be specific)

1. How many rows and columns are in the Diamonds dataset?
2. What is the mean value of *price* in the dataset?
3. What were your baseline (mean-predictor) validation *MAE* and validation *RMSE*? Report both values rounded to 2 decimals.
4. For your full-feature linear regression model (with preprocessing), what were validation *MAE* and validation *RMSE*? Report both values rounded to 2 decimals.
5. List the nonlinear engineered features you created. Did adding them improve validation *RMSE* relative to the full-feature model without engineering? Answer yes/no and report the two RMSE values (2 decimals).
6. In your greedy removal trace, at which step did validation *RMSE* reach its minimum? Provide (a) the step number, (b) the minimum validation

RMSE value (2 decimals), and (c) the number of features remaining at that step.

7. Name the first three features removed by the greedy procedure (steps 1–3). Why might these features have been the easiest to remove without harming validation error?
8. What stopping criterion did you use (keep N, percent threshold, or elbow judgment)? State it clearly and list your final selected feature set.
9. What are the final model's *test MAE* and *test RMSE*? Report both values to 2 decimals and compare them to the validation values at your chosen stopping point (better, worse, or similar?).
10. Short reflection (3–5 sentences): Why do we use a validation set during greedy feature removal and reserve the test set for one final evaluation? What would go wrong if we repeatedly used the test set to choose features?

Diamonds Predictive Practice Answers

These answers were computed using the Diamonds dataset and a predictive regression workflow with a 60/20/20 split (train/validation/test) using *random_state=42*. The model was a scikit-learn pipeline: numeric median imputation + standard scaling; categorical most-frequent imputation + one-hot encoding; then *LinearRegression*. Greedy backward feature removal was performed using validation-set *RMSE* as the primary criterion (tie-break on *MAE*).

1. The Diamonds dataset contains *53940* rows and *10* columns.

2. The mean value of *price* is *3932.7997*.
3. (Baseline) Predicting the training-mean price for every observation yields test-set *MAE = 3033.0615* and *RMSE = 3967.0696* (with R^2 = *-0.0000* up to rounding).
4. (Greedy removal, first step) The first feature removed by the greedy rule was *y* (diamond width), because removing it produced the lowest validation error among all single-feature removals at that step.
5. (Stopping point) Both validation *MAE* and validation *RMSE* reached their minimum at *step 1* (after removing *y*), so a reasonable stopping point is immediately after that first removal.
6. (Selected feature set at the stop) The remaining raw input features were *carat*, *cut*, *color*, *clarity*, *depth*, *table*, *x*, and *z* (i.e., all predictors except *y*).
7. (Final predictive performance) After selecting the stopping point, refitting the pipeline on the combined training+validation data, and evaluating once on the untouched test set, the model achieved *MAE = 806.6730*, *RMSE = 1250.6809*, and R^2 *= 0.9197* on the test set.
8. (Interpretation) The selected model dramatically outperforms the mean-baseline because it uses meaningful predictors (carat, cut, color, clarity, and dimensions) that explain most of the systematic variation in price, reducing both average error (MAE) and large-miss error (RMSE).

Case #2: Red Wine Quality Dataset (Predictive Feature Selection)
This practice uses the **Red Wine Quality** dataset (*winequality-red.csv*). You will extend your earlier multiple linear regression work by applying a **predictive modeling workflow** based on train/validation/test splits and *greedy backward feature removal*.

Dataset attribution: This dataset originates from the UCI Machine Learning Repository (Wine Quality Data Set) and was published by Cortez et al. in "Modeling wine preferences by data mining from physicochemical properties" (Decision Support Systems, 2009). It contains physicochemical measurements of red wines along with a sensory quality score.

The red wine quality dataset is available in the prior chapter if you need it.

In this chapter, you are practicing **MLR for predictive inference**. Unlike causal modeling, your goal is not to interpret coefficients or test hypotheses, but to build a model that generalizes well to new data by minimizing *out-of-sample prediction error*.

Tasks

- Inspect the dataset: rows, columns, data types, and summary statistics for *quality*.
- Create a label vector *y* using *quality* and a feature matrix *X* using all remaining numeric predictors.
- Split the data into three sets: training (60%), validation (20%), and test (20%) using fixed random seeds.
- Compute a baseline model that predicts the training-set mean of *quality* for every observation. Evaluate its MAE, RMSE, and R^2 on the test set.
- Build a preprocessing + linear regression pipeline that standardizes numeric features and fits an MLR model.
- Apply greedy backward feature removal using validation-set RMSE (tie-break using MAE) to generate a full removal trace.
- Plot validation MAE and RMSE across removal steps and identify a reasonable stopping point.

- Refit the final model using the selected feature set on the combined training + validation data.
- Evaluate the final model once on the untouched test set.

Analytical questions (answers should be specific)

1. How many rows and columns are in the Red Wine Quality dataset?
2. What is the mean value of *quality*?
3. (Baseline) What are the test-set MAE, RMSE, and R^2 when predicting the training-set mean for every observation?
4. (Greedy removal) Which feature is removed first by the greedy backward procedure?
5. (Stopping point) At which step do validation MAE and RMSE reach their minimum values?
6. (Selected features) Which raw input features remain at the chosen stopping point?
7. (Final model) What are the test-set MAE, RMSE, and R^2 of the final selected model?
8. (Interpretation) In 2–3 sentences, explain why the selected model outperforms the baseline mean predictor.

Red Wine Quality Predictive Practice Answers

These answers were computed by loading *winequality-red.csv*, splitting the data into train/validation/test sets (60/20/20 with *random_state=42*), fitting a predictive regression pipeline (impute median + standardize + linear

regression), and using greedy backward feature removal based on validation-set error (MAE and RMSE).

1. The Red Wine Quality dataset contains *1599* rows and *12* columns.
2. The mean value of *quality* is *5.6360*.
3. (Baseline) Predicting the training-mean quality for every observation yields test-set *MAE = 0.6371* and *RMSE = 0.8282* (with $R^2 = 0.0000$ up to rounding).
4. (Greedy removal, first step) The first feature removed by the greedy rule was *fixed acidity*, because removing it produced the lowest validation error among all single-feature removals at that step.
5. (Stopping point) Validation *MAE* reached its minimum at *step 4*, and validation *RMSE* also reached its minimum at *step 4* (after removing *volatile acidity* at that step), so a reasonable stopping point is *step 4*.
6. (Selected feature set at the stop) The remaining raw input features were *citric acid*, *chlorides*, *free sulfur dioxide*, *total sulfur dioxide*, *pH*, *sulphates*, and *alcohol*.
7. (Final predictive performance) After selecting the stopping point, refitting the pipeline on the combined training+validation data, and evaluating once on the untouched test set, the model achieved *MAE = 0.5149*, *RMSE = 0.6592*, and $R^2 = 0.3659$ on the test set.
8. (Interpretation) The selected model outperforms the mean-baseline because it uses informative physicochemical predictors (especially *alcohol*, *sulphates*, acidity-related measures, and sulfur dioxide measures) that capture systematic differences in quality; greedy removal reduces noise features that do not improve validation-set MAE/RMSE.

Case #3: Bike Sharing Daily Dataset (Predictive Regression)

This practice uses the **Bike Sharing** daily dataset (*day.csv*). You will build a predictive multiple linear regression model for total daily rentals (*cnt*) using the full predictive workflow from Chapter 11, including train/validation/test splits, preprocessing pipelines, baseline comparison, feature engineering, and greedy backward feature removal.

Dataset attribution: This dataset is distributed as part of the Bike Sharing Dataset hosted by the UCI Machine Learning Repository (Fanaee-T and Gama). It includes daily rental counts and weather/context variables derived from the Capital Bikeshare system in Washington, D.C. You will use the *day.csv* file provided with your course materials.

The bike sharing daily dataset is available in the prior chapter if you need it.

Important modeling note: Do not include *casual* or *registered* as predictors because they directly sum to *cnt* and would leak the answer into the model.

Goal: Build a predictive regression model that minimizes out-of-sample error (MAE and RMSE), not one that maximizes statistical significance or interpretability.

Tasks

- Inspect the dataset: number of rows, number of columns, and summary statistics for *cnt*.
- Define predictors using the following raw features: *season*, *yr*, *mnth*, *holiday*, *weekday*, *workingday*, *weathersit*, *temp*, *atemp*, *hum*, and *windspeed*.
- Split the data into training (60%), validation (20%), and test (20%) sets using random_state = 42.

- Construct a preprocessing pipeline that imputes missing values, standardizes numeric variables, and one-hot encodes categorical variables.
- Compute a baseline model that predicts the training-set mean of *cnt* for all observations and evaluate its test-set MAE, RMSE, and R^2.
- Apply greedy backward feature removal using validation-set MAE and RMSE to determine which features to remove and where to stop.
- Refit the final selected model on the combined training + validation data and evaluate it once on the test set.

Analytical questions (answers should be specific)

1. How many rows and columns are in the Bike Sharing daily dataset?
2. What is the mean value of *cnt*?
3. (Baseline) What are the test-set MAE, RMSE, and R^2 when predicting the training-set mean for every observation?
4. (Greedy removal) Which feature is removed first by the greedy backward procedure?
5. (Stopping point) At which step do validation MAE and RMSE reach their minimum values?
6. (Selected features) Which raw input features remain at the chosen stopping point?
7. (Final model) What are the test-set MAE, RMSE, and R^2 of the final selected model?
8. (Interpretation) In 2–3 sentences, explain why the selected model outperforms the baseline mean predictor.

Bike Sharing (Chapter 11) Practice Answers

These answers were computed using the Chapter 11 predictive workflow: (1) a 60/20/20 split (train/validation/test) with *random_state=42*, (2) a preprocessing pipeline that standardizes numeric predictors and one-hot encodes categorical predictors (*season*, *mnth*, *weekday*, *weathersit*), and (3) greedy backward feature removal based on validation-set *MAE* and *RMSE*.

1. The Bike Sharing daily dataset contains *731* rows and *16* columns.
2. The mean value of *cnt* is *4504.3488*.
3. (Baseline) Predicting the training-set mean *cnt* for every observation yields test-set *MAE = 1711.9909* and *RMSE = 2022.1728* (with R^2 = *-0.0198*).
4. (Greedy removal, first step) The first raw input feature removed by the greedy rule was *weekday*, because removing it produced the lowest validation error among all single-feature removals at that step.
5. (Stopping point) Both validation *MAE* and validation *RMSE* reached their minimum at *step 4* in this run.
6. (Selected features at the stop) The remaining raw input features were *season*, *yr*, *mnth*, *holiday*, *weathersit*, *temp*, and *windspeed*.
7. (Final predictive performance) After selecting the stopping point, refitting the pipeline on the combined training+validation data, and evaluating once on the untouched test set, the model achieved *MAE = 613.3657*, *RMSE = 825.0000*, and R^2 = *0.8303* on the test set.
8. (Interpretation) The selected model strongly outperforms the mean-baseline because it uses real predictive signals (seasonality, year trend, weather conditions, and temperature/wind effects) that explain

systematic variation in daily rentals. Removing weaker features reduces noise and can improve generalization, lowering both average error (MAE) and large-miss error (RMSE).

11.12Assignment

Complete the assignment below:

This assessment can be taken online.